

```
613121 HARI KANT GAUTAM Male
613168 SANGRAM RAY Male
613177 S.K. HAFIZUL ISLAM Male
613183 KAILASH KUMAR Female
613216 BRIJENDRA KUMAR Female
613238 DAMODAR REDDY EDLA Female
613240 M.RAJASEKHAR REDDY Tran
613122 SUNIL KUMAR SHARMA Male
613146 RAJIV CHOUDHARY Male
```

Another useful filtering tool is the *CONVERT()* function. It is used to convert one data type to another. For example, to convert a string date field value to the MySQL date format YYYY-MM-DD, *CONVERT* can be used as follows:

```
SELECT CONVERT('2014-02-28 08:14:57',
DATETIME);
```

Result: '2014-02-28 08:14:57'

Data processing and visualisation

After collecting, arranging and cleaning data, we are now ready to process it. The goal of processing the data is to extract meaningful information from it. Proper analysis provides insights into the business process and helps the management to take proper decisions. Here I have considered the Northwind database for analysis. As I have already discussed data analysis using SQL, I will extend the data analysis paradigm from MySQL to R, so that readers can get an idea of the full potential of a data analysis platform.

Data can be exported from MySQL to R, as follows:

```
library(RMySQL)
mydb = dbConnect(MySQL(), user='root',
password='root', dbname='northwind')

rs = dbSendQuery(mydb, "select * from
invoices")

data = fetch(rs,-1)
```

Exporting data to R

The R library, RMySQL, supports a full range of simple and complex SQL queries. For example, if you need to analyse data about the orders received by your organisation, with all the details from other tables, you can write an SQL query as follows:

```
sqlstr="
SELECT
    o.OrderID
    , p.ProductName
    , p.ProductID
    , od.UnitPrice
    , od.Quantity
    , od.UnitPrice * od.Quantity AS
subtotal
    , c.CustomerID
    , s.SupplierID
FROM orderdetails AS od

LEFT JOIN Orders AS o ON od.OrderID =
o.OrderID

LEFT JOIN Products AS p ON p.ProductID
= od.ProductID

LEFT JOIN Customers AS c on
c.CustomerID = o.CustomerID
LEFT JOIN Suppliers AS s ON
s.SupplierID = p.SupplierID; "
```

And we can execute the query with R as:

```
rs = dbSendQuery(mydb, sqlstr)
data = fetch(rs,-1)

> head(data)
```

Visualisation

Let us look at an example of data analysis and visualisation. To get a frequency distribution of ordered products, a merger of the order details and products table can be

done as follows:

```
>>>sqlstr="
select a.ProductId,b.ProductName,(a.
ProductId) As Frequency
from orderdetails as a
inner join products as b on
b.ProductID=a.ProductID
group by ProductID;"
rs = dbSendQuery(mydb, sqlstr)
data = fetch(rs,-1)
```

Since the query returns three fields, i.e., ProductID, ProductName and Frequency, a histogram of the frequency distribution of products can be written in R as shown below:

```
>>>names(data)
[1] "ProductId" "ProductName"
"Frequency"
hist(data$Frequency,col = "green",border
= "red", xlim = c(1,77), main='Histogram
of Products',xlab="Products",
ylab="Frequency")
```

Here are some simple tips to perform a few common data analysis tasks.

- To get the top ten results, one can sort the record sets with DESC and LIMIT the report for the top ten records.
- To get the statistical analysis of different fields of sales and marketing data, one can use statistical functions like *AVG()*, *STD()* and *VER()*.
- Combining data from invoices, orders, customers and employees can produce exhaustive reports to meet different requirements.
- Country and city-wise orders and shipments can be analysed by using groups like ShipCountry and ShipCity for the ordered products.
- To calculate the total revenue trend, records can be grouped on the basis of years and months. **END** 

 By: Dipankar Ray

The author is a member of IEEE, IET (UK), and has more than 20 years of experience in open source versions of the UNIX operating system.